or should("be.visible"). If the assertion needs an expected value, the way have.class does, then that value is the second argument. In a few cases, there are two arguments, so you'd add both arguments, like so: should("have.attr", "href", "/tickets").

In the argument case, you can also use the its command to coax the variable from the selected object. The its command applies a property getter to the currently yielded elements, so you could do cy.get(".thing").its("class").should("include", "thing") as a synonym of cy.get(".thing").should("have.class", "thing").

You can chain multiple should commands after each other, and if you want, you can use and as a synonym of should, as in should("have.class", "foo").and("be.visible").

The should command can also take a function. Within the function you can do assertions using regular Chai syntax, involving expect—the yielded argument is the set of elements:

```
cy.get(".tickets").should(($tickets) => {
  expect($tickets).to.have.length(10)
  expect($tickets.find("is-visible)).to.have.length(3)
}
```

You can also use expect syntax inside any then function in a Cypress command.

Stubs

Cypress uses the Sinon library for mock and stub behavior.[85] I'm not going to get into all the details of Sinon here, but it does provide the ability to define stub methods cy.stub(foo, 'bar').returns('baz') or cy.stub(obj, 'method').resolves('foo') and then assertions like expect(foo.bar).to.be.calledOnce.

You can also control the clock with cy.clock(). Once you've called cy.clock(), you can move the clock forward with cy.tick(milliseconds).

Maybe more usefully, you can also simulate a server and request data. To start this process you call cy.server(). After turning on a cy.server(), you can direct that server to produce canned responses to routes with the cy.route() command: